Table 3. Base Agent evaluation

Opponent	Games	Wins	Draws	Losses	Win rate	Draw rate	Loss rate
bots used in training							
coacAI	300	288	6	6	0.96	0.02	0.02
mayari	300	202	16	82	0.673	0.053	0.273
Built-in bots							
workerRushAI	300	300	0	0	1	0	0
passiveAI	300	299	1	0	0.997	0.003	0
lightRushAI	300	294	0	6	0.98	0	0.02
randomAI	300	297	3	0	0.99	0.01	0
randomBiasedAI	300	237	63	0	0.79	0.21	0
Other bots							
rojo	300	243	41	16	0.81	0.137	0.053
mixedBot	300	130	67	103	0.433	0.223	0.343
izanagi	300	215	36	49	0.717	0.12	0.163
tiamat	300	279	0	21	0.93	0	0.07
droplet	300	203	70	27	0.677	0.233	0.09
guidedRojoA3N	300	224	71	5	0.747	0.237	0.017
naiveMCTSAI	300	2	290	8	0.007	0.967	0.027
total	4200	3213	664	323	0.765	0.158	0.077

Table 4. Main Agent evaluation

Opponent	Games	Wins	Draws	Losses	Win rate	Draw rate	Loss rate
bots used in training							
BaseAgent	100	89	9	2	0.89	0.09	0.02
coacAI	100	100	0	0	1	0	0
mayari	100	100	0	0	1	0	0
Built-in bots							
workerRushAI	100	35	10	55	0.35	0.1	0.55
passiveAI	100	100	0	0	1	0	0
lightRushAI	100	100	0	0	1	0	0
randomAI	100	100	0	0	1	0	0
randomBiasedAI	100	97	3	0	0.97	0.03	0
Other bots							
rojo	100	100	0	0	1	0	0
mixedBot	100	57	23	20	0.57	0.23	0.2
izanagi	100	70	23	7	0.7	0.23	0.07
tiamat	100	99	0	1	0.99	0	0.01
droplet	100	62	16	22	0.62	0.16	0.22
guidedRojoA3N	100	87	13	0	0.87	0.13	0
naiveMCTSAI	100	2	98	0	0.02	0.98	0
total	1500	1198	195	107	0.799	0.130	0.071

Table 5. Difference in evaluation results between the Main Agent and the Base Agent

Opponent	Win rate	Draw rate	Loss rate
bots used in training			
coacAI	0.04	-0.02	-0.02
mayari	0.327	-0.053	-0.273
Built-in bots			
workerRushAI	-0.65	0.1	0.55
passiveAI	0.003	-0.003	0
lightRushAI	0.02	0	-0.02
randomAI	0.01	-0.01	0
randomBiasedAI	0.18	-0.18	0
Other bots			
rojo	0.19	-0.137	-0.053
mixedBot	0.137	0.007	-0.143
izanagi	-0.017	0.11	-0.093
tiamat	0.06	0	-0.06
droplet	-0.057	-0.073	0.13
guidedRojoA3N	0.123	-0.107	-0.017
naiveMCTSAI	0.013	0.013	-0.027

Table 6. Example for Main Agent without unit focus evaluation

Opponent	Games	Wins	Draws	Losses	Win rate	Draw rate	Loss rate
bots used in training							
BaseAgent	100	85	15	0	0.85	0.15	0
coacAI	100	100	0	0	1	0	0
mayari	100	100	0	0	1	0	0
Built-in bots							
workerRushAI	100	31	6	63	0.31	0.06	0.63
passiveAI	100	100	0	0	1	0	0
lightRushAI	100	100	0	0	1	0	0
randomAI	100	100	0	0	1	0	0
randomBiasedAI	100	91	9	0	0.91	0.09	0
Other bots							
rojo	100	100	0	0	1	0	0
mixedBot	100	45	22	33	0.45	0.22	0.33
izanagi	100	68	17	15	0.68	0.17	0.15
tiamat	100	94	0	6	0.94	0	0.06
droplet	100	45	33	22	0.45	0.33	0.22
guidedRojoA3N	100	64	36	0	0.64	0.36	0
naiveMCTSAI	100	0	100	0	0	1	0
total	1500	1123	238	139	0.749	0.159	0.093

Table 7. Evaluation after PPO finetuning

Opponent	Games	Wins	Draws	Losses	Win rate	Draw rate	Loss rate
bots used in training							
BaseAgent	100	92	8	0	0.92	0.08	0
coacAI	100	100	0	0	1	0	0
mayari	100	100	0	0	1	0	0
workerRushAI	100	100	0	0	1	0	0
Other built-in bots							
passiveAI	100	100	0	0	1	0	0
lightRushAI	100	100	0	0	1	0	0
randomAI	100	100	0	0	1	0	0
randomBiasedAI	100	98	2	0	0.98	0.02	0
Other bots							
rojo	100	100	0	0	1	0	0
mixedBot	100	59	24	17	0.59	0.24	0.17
izanagi	100	68	29	3	0.68	0.29	0.03
tiamat	100	99	0	1	0.99	0	0.01
droplet	100	66	20	14	0.66	0.2	0.14
guidedRojoA3N	100	84	16	0	0.84	0.16	0
naiveMCTSAI	100	2	97	1	0.02	0.97	0.01
total	1500	1268	196	36	0.845	0.131	0.184

Table 8. Hyperparameters and control parameters used for the LT setup

Category	Parameter	Value	Description
LT Setup			
League	num_main_exploiters	4	Number of Main Exploiters
League	num_league_exploiters	2	Number of League Exploiters
League	selfplay_ready_save_interval	$5 \cdot 10^5$	Step interval for potential league checkpoints (checkpoint if the win rate is sufficiently high)
League	main_selfplay_save_interval	$6 \cdot 10^6$	Step interval for forced Main Agent checkpoints
League	main_exploiter_selfplay_save_interval	$9 \cdot 10^6$	Step interval for forced Main Exploiter checkpoints
League	league_exploiter_selfplay_save_interval	$9 \cdot 10^6$	Step interval for forced League Exploiter checkpoints
League	main_winrate_threshold	0,7	Win rate threshold for Main Agent checkpoints
League	main_exploiter_winrate_threshold	0,7	Win rate threshold for Main Exploiter checkpoints
League	league_exploiter_winrate_threshold	0,7	Win rate threshold for League Exploiter checkpoints
League	main_exploiter_vs_main_winrate_threshold	0,3	Threshold for Main Exploiters against the Main Agent
PFSP	pfsp_min_prob_factor	0,25	Minimum probability factor added to PFSP
PFSP	main_PFSP_prob	0,5	Probability of using PFSP with historicals for the Main Agent
PFSP	main_SP_prob	0,35	Probability of SP for the Main Agent
Environments and Training Scope			
Environments	num_bot_envs	5	Initial number of bot environments.
Environments	min_num_bot_envs	5	Lower bound for the number of bot environments
Environments	max_num_bot_envs	10	Upper bound for the number of bot environments
Environments	min_bot_winrate	0,955	Lower win rate threshold for bot environment adjustment (if crossed num_bot_envs == max_num_bot_envs)
Environments	max_bot_winrate	0,98	Upper win rate threshold for bot environment adjustment (if crossed num_bot_envs == min_num_bot_envs)
Environments	num_main_envs	25	Number of parallel environments for the Main Agent
Environments	num_envs_per_main_exploiters	25	Number of environments per Main Exploiter
Environments	num_envs_per_league_exploiters	25	Number of environments per League Exploiter
Training	total_timesteps	200 000 000	Total number of training steps
Training	checkpoint_end_buffer_steps	$3 \cdot 10^6$	Exploiters cannot checkpoint checkpoint_end_buffer_steps steps bevor reaching total_timesteps
Reward Function			
Reward	attack_reward_weight	0,0	Weight of the attack reward
Reward	rewardscore	0,002	Scaling factor of delta score.
PPO Hyperparameters (Main Agent)			
PPO Main Agent	PPO_learning_rate	$5 \cdot 10^{-5}$	Learning rate of the Main Agent
PPO Main Agent	num_steps	512	Number of steps per rollout
PPO Main Agent	gamma	1,0	Discount factor
PPO Main Agent	gae_lambda	0,95	Generalized Advantage Estimation (GAE) parameter
PPO Main Agent	ent_coef	0,01	Entropy regularization coefficient
PPO Main Agent	vf_coef	0,15	Weight of the value loss
PPO Main Agent	max_grad_norm	0,5	Gradient clipping threshold
PPO Main Agent	clip_coef	0,15	PPO clipping coefficient
PPO Main Agent	update_epochs	4	Number of update epochs per PPO cycle.
PPO Main Agent	kle_stop	true	Early stopping when KL divergence becomes too large
PPO Main Agent	kle_rollback	false	No rollback when the KL threshold is exceeded
PPO Main Agent	target_kl	0,01	Target KL divergence value
PPO Main Agent	kl_coeff	0,3	Coefficient of the KL penalty term
PPO Main Agent	norm_adv	true	Advantages are normalized
PPO Main Agent	anneal_lr	true	Learning rate annealing is enabled
PPO Main Agent	clip_vloss	true	Value loss clipping is enabled
PPO Hyperparameters (exploiters) (only differences to Main Agent hyperparameters)			
PPO Exploiter	exploiter_ent_coef	0,015	Entropy regularization coefficient of the exploiters
PPO Exploiter	exploiter_vf_coef	0,1	Weight of the value loss for the exploiters

Some Parameters (e. g. model paths, project names, or logging names) were omitted for clarity.

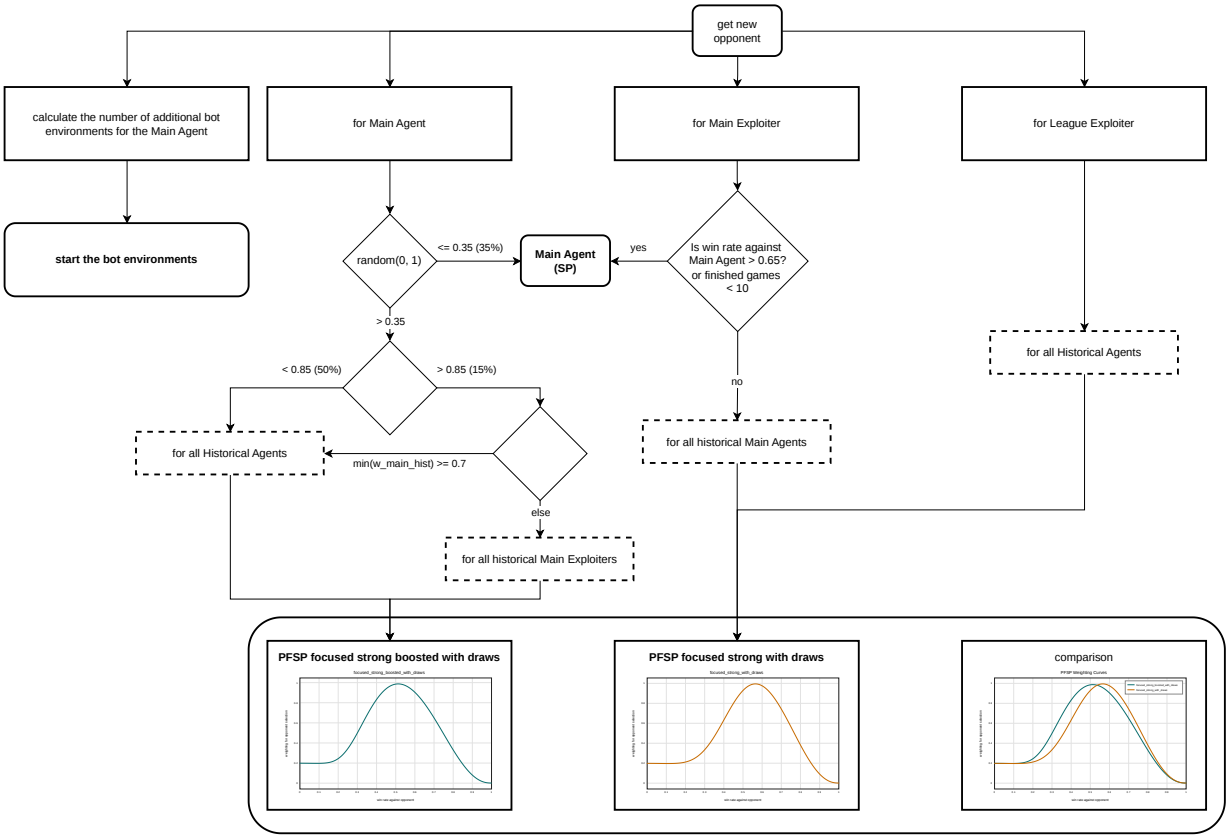

Figure 7. Matchmaking process for LT. Dashed boxes are candidate opponent sets, while the percentages on the arrows are fixed sampling probabilities. The PFSP curve gives the weighting for the opponent win rate. We define `w_main_hist` as the set of win rates of the Main Agent against all historical Main Exploiters.

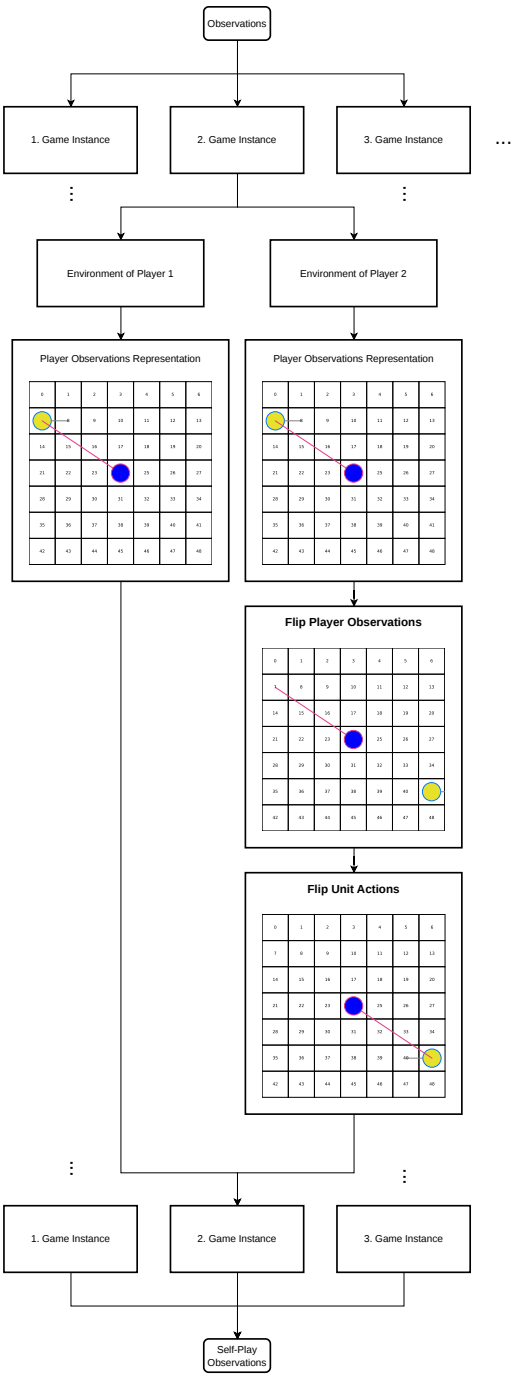

Figure 8. Observation adjustment for player 2 in SP. To rotate the observations, the GridNet representation was flipped for each player-2 environment. Afterwards, the unit actions in the observation were flipped individually.

References

[Dhariwal et al., 2017] Dhariwal, P., Hesse, C., Klimov, O., Nichol, A., Plappert, M., Radford, A., Schulman, J., Sidor, S., Wu, Y., and Zhokhov, P. (2017). Openai baselines. https://github.com/openai/baselines. 3

[Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). Impala: scalable distributed deep-rl with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1406–1415. 3, 4

[Goodfriend, 2024] Goodfriend, S. (2024). Raisocketai: The first deep reinforcement learning agent to win the ieee microrts competition. In *2024 IEEE Conference on Games (CoG)*, pages 1–8. 3, 4, 5, 6, 10

[Han et al., 2019] Han, L., Sun, P., Du, Y., Xiong, J., Wang, Q., Sun, X., Liu, H., and Zhang, T. (2019). Grid-wise control for multi-agent reinforcement learning in video game AI. In Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2576–2585. PMLR. 3, 5

[Heinrich et al., 2015] Heinrich, J., Lanctot, M., and Silver, D. (2015). Fictitious self-play in extensive-form games. In Bach, F. and Blei, D., editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 805–813. 2, 7

[Huang et al., 2021] Huang, S., Ontanón, S., Bamford, C., and Grela, L. (2021). Gym-microrts: Toward affordable full game real-time strategy games research with deep reinforcement learning. *CoRR*, abs/2105.13807. 2, 3, 5, 6, 10

[Huft, 2025] Huft, M. (2025). Game playing agents in microrts. 2, 3, 5, 6, 10, 11, 12

[Martin and Jhala, 2021] Martin, D. and Jhala, A. (2021). Beclone: Behavior cloning with inference for real-time strategy games. In *Joint Proceedings of the AIIDE 2021 Workshops*. 2, 4, 6

[Ontañón, 2013] Ontañón, S. (2013). The combinatorial multi-armed bandit problem and its application to real-time strategy games. In *Proceedings of the Ninth Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 9, pages 58–64. AAAI. 3, 10

[Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347. 2, 6

[Silver et al., 2018] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362:1140–1144. 6

[Sutton and Barto, 1998] Sutton, R. S. and Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. The MIT Press. 4

[Torabi et al., 2018] Torabi, F., Warnell, G., and Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*. 2, 6

[Vinyals et al., 2019] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., and Silver, D. (2019). Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354. 2, 4, 6, 7, 8, 9, 14, 15

Erklärung

(???) GPT!!!